IIT Bombay Invigilation Scheduler: Complete Line-by-Line Code Analysis

Document Type: Line-by-Line Complete Technical Code Walkthrough
Target Codebase: `invigilation_scheduler.py` (1293 Lines)
Scope: Detailed explanation of all modules, logic constructs, and algorithms without omission.

Section 1: Imports and Core Library Setup (Lines 1–11)

Imports the standard library dependencies for enums, time benchmarking, JSON serialization, OS paths, random sampling, and mathematical operations. It also imports type hinting generics and the dataclass utilities.

```
1  import enum
2  import time
3  import json
4  import sys
5  import os
6  import random
7  import math
8  from dataclasses import dataclass, field
9  from typing import List, Dict, Set, Tuple, Optional, Any
10 from datetime import datetime
11
```

Lines / Code	Detailed Functional & Logic Explanation
Lines 1	Imports the `enum` module, which is used to define structured, readable string enums for configurations (like `ExamType`).
Lines 2	Imports the `time` module, which is used to benchmark the execution time of the scheduling algorithms.
Lines 3	Imports the `json` module, which is used to read input files and print output payloads.
Lines 4	Imports the `sys` module, which is used to read command line arguments and execute clean exits on failure.
Lines 5	Imports the `os` module, which is used to perform filesystem checks (like verifying if a config file path exists).
Lines 6	Imports the `random` module, which is the core engine for generating random candidate decisions inside the Local Search optimizer.
Lines 7	Imports the `math` module, which is used for mathematical operations like calculating absolute values and exponents.
Lines 8	Imports `dataclass` and `field` from `dataclasses`. This allows the application to represent domain objects (Faculty, Sessions) cleanly without verbose constructor code.
Lines 9	Imports typing generics (`List`, `Dict`, `Set`, `Tuple`, `Optional`, `Any`) to allow strict static typing checks.

Lines / Code	Detailed Functional & Logic Explanation
Lines 10	Imports `datetime` from `datetime` to capture timestamp information for logs and reports.

Section 2: Domain Data Models & Dataclasses (Lines 12–89)

Defines the core data structures of the scheduling domain. It covers the ExamType enum, FacultyCategory weights, Faculty properties (with conflict slots and overrides), Sessions, Historical Records, and final result wrappers.

```
# ============================================================================
# 1. DATA MODELS
# ============================================================================

class ExamType(str, enum.Enum):
    MIDSEM = "midsem"
    ENDSEM = "endsem"

@dataclass
class FacultyCategory:
    name: str  # e.g., "Professor", "Associate Professor", "Assistant Professor"
    ratio_weight: float  # The ratio weight representing workload priority (e.g., 2, 3, 4)

@dataclass
class Faculty:
    id: str
    name: str
    category_name: str  # Must match a category name in FacultyCategory dict
    pg_timetable_blocks: List[str] = field(default_factory=list)  # Session IDs with PG class conflicts (applicable during midsem)
    availability_overrides: List[str] = field(default_factory=list)  # Session IDs where faculty is unavailable

@dataclass
class Session:
    id: str  # e.g., "D11", "D12", ..., "D62"
    day: int  # 1 to 6 (Monday = 1, Saturday = 6)
    session_num: int  # 1 (FN) or 2 (AN)
    label: str  # e.g., "Monday FN"
    required_invigilators: int
    day_weight: float = 1.0  # 1.0 for weekday, 1.5 for Saturday
    duration_hours: float = 2.0  # e.g., 2.0 for midsem, 3.0 for endsem

@dataclass
class HistoricalRecord:
    faculty_id: str
    previous_imbalance: float  # +n for overload of n hours, -m for underload of m hours, 0 for neutral

@dataclass
class AllocationInput:
    faculty_list: List[Faculty]
    categories: Dict[str, FacultyCategory]
    sessions: List[Session]
    history: List[HistoricalRecord] = field(default_factory=list)
    exam_type: ExamType = ExamType.MIDSEM
    category_ratio_mode: str = "target_load_scaling"

@dataclass
class FacultyReport:
    faculty_id: str
    name: str
    category_name: str
    assigned_sessions: List[str]
    assigned_hours: float
    assigned_weighted_load: float
    target_load: float
    overload: float
    underload: float
    historical_imbalance: float
```

```
    cumulative_imbalance: float
    impact_status: str  # "IMPROVED", "WORSENED", or "NEUTRAL"
    selection_explanations: Dict[str, str] = field(default_factory=dict)  # Explanation per
assigned session ID

@dataclass
class SessionAllocation:
    session_id: str
    assigned_faculty_ids: List[str]

@dataclass
class AllocationResult:
    success: bool
    schedule: List[SessionAllocation]
    faculty_summaries: List[FacultyReport]
    jains_fairness_index: float
    gini_coefficient: float
    feasibility_report: str
    conflict_report: List[str]
    history_impact_report: str

# ============================================================================
```

Lines / Code	Detailed Functional & Logic Explanation
Lines 16-18	Defines `ExamType` as a string enum (`str, enum.Enum`). By inheriting from `str`, the enum values act as native strings, meaning they can be serialized directly to and from JSON without requiring a custom JSON encoder.
Lines 20-23	Defines the `FacultyCategory` dataclass. The `ratio_weight` field defines the workload proportions (e.g. if a Professor has weight 2.0 and an Assistant Professor has 4.0, the Assistant Professor should ideally carry double the workload).
Lines 25-31	Defines the `Faculty` dataclass. Crucially, `pg_timetable_blocks` and `availability_overrides` default to `field(default_factory=list)`. This prevents the common Python bug where all object instances share a single mutable list reference in memory if set as a default parameter.
Lines 33-41	Defines the `Session` dataclass. It maps an exam slot's day, number, required invigilator count, and duration. It also includes `day_weight` (which defaults to 1.5 on Saturdays to reward or penalize weekend shifts).
Lines 43-46	Defines `HistoricalRecord`. It holds the `previous_imbalance` in hours from previous semesters (positive means overloaded, negative means underloaded).
Lines 48-56	Defines the `AllocationInput` dataclass. This groups all parsed configuration inputs together into a single object, simplifying parameter passing to the solver.
Lines 58-71	Defines `FacultyReport`. This tracks a faculty member's assignments, hours, weighted load, targets, and history impact status (`IMPROVED`, `WORSENED`, or `NEUTRAL`) for final reports.
Lines 73-76	Defines `SessionAllocation`. It maps a `session_id` directly to a list of assigned `faculty_ids` representing who invigilates that exam.
Lines 78-88	Defines the final `AllocationResult` dataclass. It includes success flags, lists of allocations, fairness scores, constraint violations, and diagnostic reports.

Section 3: Metrics & Workload Math (Lines 90–196)

Contains mathematical utility functions that compute session weighted workloads, target loads based on ratio modes, Jain's Fairness Index, and the Gini Coefficient.

```
# 2. METRIC CALCULATIONS
# ============================================================================

def calculate_session_weighted_hours(session: Session) -> float:
    """Computes the weighted hours required for a single slot in a session."""
    return session.duration_hours * session.day_weight

def calculate_total_required_weighted_hours(sessions: List[Session]) -> float:
    """Computes the sum of weighted hours required across all sessions and slots."""
    total = 0.0
    for s in sessions:
        total += s.required_invigilators * calculate_session_weighted_hours(s)
    return total

def calculate_target_loads(
    faculty_list: List[Faculty],
    categories: Dict[str, FacultyCategory],
    sessions: List[Session],
    ratio_mode: str = "target_load_scaling",
    custom_raw_targets: Optional[Dict[str, float]] = None
) -> Dict[str, float]:
    """
    Computes the target weighted workload (in hours) for each faculty member.

    ratio_mode options:
      - "target_load_scaling": Target load for each faculty is proportional to their
        category ratio weight, scaled so that the sum of target loads matches the total
        required weighted hours across all sessions.
      - "raw_weights": Target load for each faculty is exactly the category ratio weight
(unscaled).
      - "hard_category_limits": Target load is scaled proportionally to ratio weight,
        and treated as a hard maximum load limit.
    """
    if not faculty_list:
        return {}

    for f in faculty_list:
        if f.category_name not in categories:
            raise ValueError(f"Category '{f.category_name}' for faculty '{f.id}' not found in
categories dict.")

    total_required = calculate_total_required_weighted_hours(sessions)

    if ratio_mode in ("target_load_scaling", "hard_category_limits"):
        sum_weights = sum(categories[f.category_name].ratio_weight for f in faculty_list)
        if sum_weights == 0:
            return {f.id: 0.0 for f in faculty_list}

        scaling_factor = total_required / sum_weights
        return {f.id: categories[f.category_name].ratio_weight * scaling_factor for f in
faculty_list}

    elif ratio_mode == "raw_weights":
        if custom_raw_targets is not None:
            return {f.id: custom_raw_targets.get(f.category_name,
categories[f.category_name].ratio_weight) for f in faculty_list}
        return {f.id: categories[f.category_name].ratio_weight for f in faculty_list}

    else:
        raise ValueError(f"Unknown ratio_mode: {ratio_mode}")
```

```
146
147 def calculate_jains_index(loads: List[float], targets: List[float]) -> float:
148     """
149     Computes Jain's Fairness Index for the given workloads normalized by target loads.
150     J = (sum(x_i))^2 / (n * sum(x_i^2)) where x_i = load_i / target_i
151     """
152     n = len(loads)
153     if n == 0:
154         return 1.0
155
156     ratios = []
157     for l, t in zip(loads, targets):
158         if t > 0:
159             ratios.append(l / t)
160         else:
161             ratios.append(1.0 if l == 0 else 0.0)
162
163     sum_ratios = sum(ratios)
164     if sum_ratios == 0:
165         return 1.0
166
167     sum_sq_ratios = sum(x ** 2 for x in ratios)
168     return (sum_ratios ** 2) / (n * sum_sq_ratios)
169
170 def calculate_gini_coefficient(loads: List[float], targets: List[float]) -> float:
171     """
172     Computes the Gini Coefficient for the given workloads normalized by target loads.
173     G = sum_{i,j} |x_i - x_j| / (2 * n * sum(x_i)) where x_i = load_i / target_i
174     """
175     n = len(loads)
176     if n == 0:
177         return 0.0
178
179     ratios = []
180     for l, t in zip(loads, targets):
181         if t > 0:
182             ratios.append(l / t)
183         else:
184             ratios.append(1.0 if l == 0 else 2.0)
185
186     sum_ratios = sum(ratios)
187     if sum_ratios == 0:
188         return 0.0
189
190     diff_sum = 0.0
191     for r_i in ratios:
192         for r_j in ratios:
193             diff_sum += abs(r_i - r_j)
194
195     return diff_sum / (2 * n * sum_ratios)
196
```

Lines / Code	Detailed Functional & Logic Explanation
Lines 93-95	Defines `calculate_session_weighted_hours`, which multiplies session duration by day weight (e.g. a 2-hour weekday session is 2.0 weighted hours, while a Saturday session is 3.0 weighted hours).

Lines / Code	Detailed Functional & Logic Explanation
Lines 97-102	Defines `calculate_total_required_weighted_hours`, which sums up the required weighted hours across all sessions and slot requirements. This sets the total workload that must be distributed.
Lines 104-145	Defines `calculate_target_loads`. It scales category ratio weights dynamically to distribute the total required hours fairly: `scaling_factor = total_required / sum_weights`. In `raw_weights` mode, it treats ratio weights directly as target hours.
Lines 147-168	Defines `calculate_jains_index`. Normalizes workloads by targets (`ratio = actual / target`) and computes Jain's index. A score of 1.0 represents perfect fairness, while 1/n represents absolute unfairness.
Lines 170-195	Defines `calculate_gini_coefficient`. Computes the Gini inequality score using a double loop to sum the absolute differences between all pairs of normalized workloads. A score of 0.0 represents perfect equality, and 1.0 represents maximum inequality.

Section 4: InvigilationSolver Init & Eligibility Checks (Lines 197–260)

Deconstructs the solver's constructor, which sets up maps, indexes, and calculates targets. It also covers the core eligibility method that dynamically checks hard constraints during scheduling.

```
# ======================================================================
# 3. CORE SCHEDULING SOLVER ENGINE
# ======================================================================

class InvigilationSolver:
    """
    Robust university invigilation schedule solver.
    Uses a hybrid approach of Greedy Initialization, Hill Climbing Local Search,
    and exact Backtracking Branch-and-Bound.

    ratio_mode options:
      - "target_load_scaling": Scale targets so their sum matches total required exam hours.
      - "raw_weights": Treat ratio weights directly as target hours.
      - "hard_category_limits": Proportional scaled targets, treated as hard maximum limits.
    """
    def __init__(self, input_data: AllocationInput, ratio_mode: Optional[str] = None,
custom_raw_targets: Optional[Dict[str, float]] = None):
        self.input_data = input_data
        self.ratio_mode = ratio_mode or input_data.category_ratio_mode or "target_load_scaling"
        self.custom_raw_targets = custom_raw_targets

        self.categories = input_data.categories
        self.faculty_map = {f.id: f for f in input_data.faculty_list}
        self.faculty_index = {f.id: i for i, f in enumerate(input_data.faculty_list)}
        self.sessions = sorted(input_data.sessions, key=lambda s: s.id)

        # Build history map
        self.history_map = {f.id: 0.0 for f in input_data.faculty_list}
        for record in input_data.history:
            if record.faculty_id in self.history_map:
                self.history_map[record.faculty_id] = record.previous_imbalance

        # Calculate target loads based on ratio weighting mode
        self.target_loads = calculate_target_loads(
            faculty_list=input_data.faculty_list,
            categories=self.categories,
            sessions=self.sessions,
            ratio_mode=self.ratio_mode,
            custom_raw_targets=self.custom_raw_targets
        )

    def _is_faculty_eligible_for_session(self, fac_id: str, session: Session, current_load:
float, ignore_pg: bool = False, ignore_overrides: bool = False) -> bool:
        """
        Validates all constraints for assigning a faculty to a session dynamically,
        including hard category limit bounds if active.
        """
        faculty = self.faculty_map[fac_id]

        # 1. PG conflict check during midsems
        if not ignore_pg and self.input_data.exam_type == ExamType.MIDSEM:
            if session.id in faculty.pg_timetable_blocks:
                return False

        # 2. Availability override check
        if not ignore_overrides and session.id in faculty.availability_overrides:
            return False

        # 3. Hard category limit check
        if self.ratio_mode == "hard_category_limits":
```

```
            w_hrs = calculate_session_weighted_hours(session)
            if current_load + w_hrs > self.target_loads[fac_id] + 1e-5:
                return False

        return True

```

Lines / Code	Detailed Functional & Logic Explanation
Lines 212-235	The `__init__` constructor maps input data, builds a `faculty_map` and a `faculty_index` for fast lookups, reads history records, and calculates target loads.
Lines 237-260	Defines `_is_faculty_eligible_for_session`. Enforces three hard constraints: 1) Blocks PG conflicts during midsems, 2) Blocks availability overrides, and 3) Blocks assignments that exceed target loads if in `hard_category_limits` mode.

Section 5: Schedule Validation Engine (Lines 261–328)

Implements validation checks to ensure that a generated schedule satisfies all constraints (e.g. coverage, daily limit of one duty, PG conflicts, overrides, and hard limits).

```
261     def validate_allocation(self, schedule: List[SessionAllocation]) -> List[str]:
262         """
263         Explicitly validates the generated schedule against all hard constraints.
264         Returns a list of validation error messages. If empty, the schedule is fully valid.
265         """
266         errors = []
267         # 1. Check session coverage
268         assigned_slots = {sa.session_id: sa.assigned_faculty_ids for sa in schedule}
269         for session in self.sessions:
270             assigned_facs = assigned_slots.get(session.id, [])
271             if len(assigned_facs) != session.required_invigilators:
272                 errors.append(
273                     f"Session Coverage Violation: Session {session.id} ({session.label})
requires "
274                     f"{session.required_invigilators} invigilators, but has {len(assigned_facs)}
assigned."
275                 )
276 
277         # 2. Check at-most-one duty per day, overrides, PG conflicts, and hard limits
278         faculty_day_duties: Dict[str, Dict[int, List[str]]] = {}
279         faculty_load: Dict[str, float] = {f.id: 0.0 for f in self.input_data.faculty_list}
280 
281         for sa in schedule:
282             session = next((s for s in self.sessions if s.id == sa.session_id), None)
283             if not session:
284                 continue
285             w_hrs = calculate_session_weighted_hours(session)
286             for f_id in sa.assigned_faculty_ids:
287                 faculty = self.faculty_map[f_id]
288                 faculty_day_duties.setdefault(f_id, {}).setdefault(session.day,
[]).append(session.id)
289                 faculty_load[f_id] += w_hrs
290 
291                 # Check PG conflicts (only for midsem)
292                 if self.input_data.exam_type == ExamType.MIDSEM:
293                     if session.id in faculty.pg_timetable_blocks:
294                         errors.append(
295                             f"PG Timetable Conflict: Faculty {faculty.name} ({f_id}) is assigned
to "
296                             f"Session {session.id} ({session.label}) despite having a PG lecture
block."
297                         )
298 
299                 # Check availability overrides
300                 if session.id in faculty.availability_overrides:
301                     errors.append(
302                         f"Availability Violation: Faculty {faculty.name} ({f_id}) is assigned to
"
303                         f"Session {session.id} ({session.label}) despite being marked
unavailable."
304                     )
305 
306         # Check daily limits (at most one duty per day)
307         for f_id, day_map in faculty_day_duties.items():
308             faculty = self.faculty_map[f_id]
309             for day, sessions in day_map.items():
310                 if len(sessions) > 1:
311                     errors.append(
312                         f"One Duty Per Day Violation: Faculty {faculty.name} ({f_id}) is
assigned to "
```

```
                        f"multiple sessions ({', '.join(sessions)}) on Day {day}."
                    )

        # Check hard category limits
        if self.ratio_mode == "hard_category_limits":
            for f_id, load in faculty_load.items():
                target = self.target_loads[f_id]
                if load > target + 1e-5:
                    faculty = self.faculty_map[f_id]
                    errors.append(
                        f"Hard Category Limit Violation: Faculty {faculty.name} ({f_id})
assigned load "
                        f"{load:.2f} hrs exceeds their hard target load of {target:.2f} hrs."
                    )

        return errors

```

Lines / Code	Detailed Functional & Logic Explanation
Lines 267-275	Validates session coverage: verifies if the number of assigned invigilators matches the required count for each session.
Lines 278-305	Loops over allocations to check for PG conflicts (for midsems) and availability overrides, logging error details for any violations.
Lines 307-314	Enforces the daily limit: checks if any faculty member is assigned to multiple sessions on the same day, logging a violation if they are.
Lines 316-325	Enforces hard category limits: checks if a faculty member's assigned workload exceeds their target load in `hard_category_limits` mode.

Section 6: Static Feasibility Engine (Lines 329–382)

Implements static checks to determine if the scheduling problem is mathematically solvable before running the solver, verifying session availability and daily capacity limits.

```
329     def check_feasibility(self) -> Tuple[bool, str]:
330         """
331         Performs static checks to determine if the scheduling problem is feasible.
332         Returns (is_feasible, diagnostic_message).
333         """
334         if not self.input_data.faculty_list:
335             return False, "Feasibility Check Failed: No faculty members provided in the input."
336         if not self.sessions:
337             return False, "Feasibility Check Failed: No exam sessions provided in the input."
338 
339         # 1. Check if each session has enough available faculty statically
340         for session in self.sessions:
341             available_faculty = self._get_available_faculty_for_session(session,
ignore_pg=False, ignore_overrides=False)
342             if len(available_faculty) < session.required_invigilators:
343                 return False, (
344                     f"Feasibility Check Failed: Session {session.id} ({session.label}) requires
"
345                     f"{session.required_invigilators} invigilators, but only
{len(available_faculty)} "
346                     f"faculty members are available due to PG class conflicts or availability
overrides."
347                 )
348 
349         # 2. Check daily unique faculty capacity
350         day_sessions: Dict[int, List[Session]] = {}
351         for session in self.sessions:
352             day_sessions.setdefault(session.day, []).append(session)
353 
354         for day, sessions_on_day in day_sessions.items():
355             day_req = sum(s.required_invigilators for s in sessions_on_day)
356             available_on_day: Set[str] = set()
357             for session in sessions_on_day:
358                 available_on_day.update(
359                     f.id for f in self._get_available_faculty_for_session(session,
ignore_pg=False, ignore_overrides=False)
360                 )
361             if len(available_on_day) < day_req:
362                 session_labels = ", ".join(s.id for s in sessions_on_day)
363                 return False, (
364                     f"Feasibility Check Failed: Day {day} (sessions: {session_labels}) requires
a total "
365                     f"of {day_req} invigilators, but only {len(available_on_day)} unique faculty
members "
366                     f"are available on this day (respecting PG classes and overrides). Each
faculty can do at most 1 duty/day."
367                 )
368 
369         return True, "Passed basic static feasibility checks."
370 
371     def _get_available_faculty_for_session(self, session: Session, ignore_pg: bool = False,
ignore_overrides: bool = False) -> List[Faculty]:
372         """Returns the list of faculty members who do not have static PG/availability conflicts
for the session."""
373         available = []
374         for f in self.input_data.faculty_list:
375             if not ignore_pg and self.input_data.exam_type == ExamType.MIDSEM:
376                 if session.id in f.pg_timetable_blocks:
377                     continue
378             if not ignore_overrides:
```

```
                if session.id in f.availability_overrides:
                    continue
            available.append(f)
        return available
```

Lines / Code	Detailed Functional & Logic Explanation
Lines 339-347	Checks if each session has enough available faculty (who have no static PG conflicts or overrides) to meet its invigilator requirement.
Lines 349-367	Enforces the daily capacity check using the Pigeonhole Principle: counts the number of unique faculty members available on each day. If a day requires a total of N slots but only has M available unique faculty (where M < N), the check fails immediately since each faculty can work at most once per day.
Lines 371-382	Defines `_get_available_faculty_for_session`, a helper function that returns all faculty members who have no PG conflicts or overrides for a given session.

Section 7: Constraint Relaxation Diagnostics (Lines 384–441)

If feasibility checks fail, this engine relaxes constraints one-by-one to pinpoint the cause of the conflict.

```
    def run_diagnostics(self) -> str:
        """
        Runs diagnostic evaluations to identify which constraints cause infeasibility by relaxing them.
        """
        import copy

        # Test if removing PG timetable conflicts makes it feasible
        input_no_pg = copy.deepcopy(self.input_data)
        for f in input_no_pg.faculty_list:
            f.pg_timetable_blocks = []
        temp_solver_no_pg = InvigilationSolver(input_no_pg, self.ratio_mode, self.custom_raw_targets)
        is_feas, msg = temp_solver_no_pg.check_feasibility()
        if is_feas:
            res = temp_solver_no_pg.solve(max_steps=5000, run_diag=False)
            if res.success:
                return (
                    "Infeasibility Cause: PG Timetable Conflicts. "
                    "The schedule becomes feasible if faculty lecture schedules are suspended or ignored."
                )

        # Test if removing availability overrides makes it feasible
        input_no_override = copy.deepcopy(self.input_data)
        for f in input_no_override.faculty_list:
            f.availability_overrides = []
        temp_solver_no_override = InvigilationSolver(input_no_override, self.ratio_mode, self.custom_raw_targets)
        is_feas, msg = temp_solver_no_override.check_feasibility()
        if is_feas:
            res = temp_solver_no_override.solve(max_steps=5000, run_diag=False)
            if res.success:
                return (
                    "Infeasibility Cause: Availability Overrides. "
                    "The schedule becomes feasible if special unavailability requests/overrides are ignored."
                )

        # Test daily limits
        day_sessions: Dict[int, List[Session]] = {}
        for session in self.sessions:
            day_sessions.setdefault(session.day, []).append(session)
        for day, sessions_on_day in day_sessions.items():
            day_req = sum(s.required_invigilators for s in sessions_on_day)
            available_on_day = set()
            for session in sessions_on_day:
                available_on_day.update(
                    f.id for f in self._get_available_faculty_for_session(session, ignore_pg=False, ignore_overrides=False)
                )
            if len(available_on_day) < day_req:
                return (
                    f"Infeasibility Cause: At-most-one duty per day constraint. "
                    f"Day {day} requires {day_req} duties, but only {len(available_on_day)} faculty are available. "
                    f"Some faculty members must be assigned multiple duties on Day {day} to cover the exams."
                )

```

```
436          return (
437              "Infeasibility Cause: Multiple interacting constraints (PG conflicts, overrides, and
daily duty limits). "
438              "The total supply of available faculty hours is insufficient to cover all requested
sessions. "
439              "Please add more faculty, reduce required invigilators, or lift availability
blocks."
440          )
441
```

Lines / Code	Detailed Functional & Logic Explanation
Lines 390-402	Deep-copies the input data, clears all PG timetable conflicts, and runs feasibility checks. If it passes, it reports that PG conflicts are the cause of the infeasibility.
Lines 404-416	Clears availability overrides and runs feasibility checks. If it passes, it reports that availability overrides are the cause of the infeasibility.
Lines 418-434	Evaluates day-wise capacity limits to identify the exact day that is short of staff.
Lines 436-440	Returns a detailed diagnostic message if the failure is caused by a combination of multiple constraints.

Section 8: Objective Function Formulation (Lines 442–470)

Defines the objective function that the solver minimizes. It sums the squared workload deviations, applies penalties for worsening imbalances, and adds large penalties for unassigned slots.

```
442     def _get_objective(self, assign_dict: Dict[str, List[str]], load_dict: Dict[str, float]) ->
float:
443         """
444         Calculates the objective function to minimize.
445         Formulation:
446           sum_f ( (H_f + A_f - T_f)^2 ) + sum_f ( worsening_penalty_f ) + unfilled_slots *
100000.0
447         """
448         obj = 0.0
449         for f_id in self.target_loads:
450             h_f = self.history_map[f_id]
451             a_f = load_dict[f_id]
452             t_f = self.target_loads[f_id]
453             c_f = h_f + a_f - t_f
454             obj += c_f ** 2
455 
456             # Strict history preservation penalty for worsening
457             if abs(h_f) > 1e-5:
458                 worsening = max(0.0, abs(c_f) - abs(h_f))
459                 obj += worsening * 100.0
460 
461         # Unassigned slot penalty (heavily penalizes unassigned slots)
462         unassigned_count = 0
463         for session in self.sessions:
464             assigned_count = len(assign_dict.get(session.id, []))
465             missing = session.required_invigilators - assigned_count
466             unassigned_count += missing
467 
468         obj += unassigned_count * 100000.0
469         return obj
470 
```

Lines / Code	Detailed Functional & Logic Explanation
Lines 449-454	Computes the workload deviation for each faculty member: `cumulative_load = history + actual - target`. It squares this difference so that larger deviations are penalized disproportionately more than smaller ones, guiding the optimizer toward balanced loads.
Lines 456-459	Applies a worsening penalty: if a faculty member's historical imbalance is made worse by their assignment, it adds a penalty of 100x the worsening amount to protect them.
Lines 461-468	Applies a large penalty (100,000 per slot) for any unfilled invigilator slots, ensuring the solver prioritizes filling exam slots over workload fairness.

Section 9: Greedy Heuristic Initialization (Lines 471–515)

Constructs an initial schedule using the Minimum Remaining Values (MRV) heuristic and Least Cost selection.

```
471      def _run_greedy_initialization(self) -> Tuple[float, Dict[str, List[str]]]:
472          """
473          Runs a quick greedy heuristic to construct a valid initial solution,
474          strictly respecting overrides, PG conflicts, daily duty limits, and hard category
targets.
475          """
476          assignment = {s.id: [] for s in self.sessions}
477          faculty_daily_duties = {f.id: set() for f in self.input_data.faculty_list}
478          faculty_weighted_load = {f.id: 0.0 for f in self.input_data.faculty_list}
479
480          # Sort sessions: saturday sessions first, and sessions with fewer available faculty
first
481          session_avail_counts = []
482          for s in self.sessions:
483              avail = self._get_available_faculty_for_session(s)
484              session_avail_counts.append((s, len(avail)))
485          sorted_sessions = [
486              x[0] for x in sorted(
487                  session_avail_counts,
488                  key=lambda x: (x[1], -x[0].day_weight, x[0].id)
489              )
490          ]
491
492          for session in sorted_sessions:
493              available = self._get_available_faculty_for_session(session)
494              # Filter dynamically for daily limits and hard category limits
495              eligible = [
496                  f for f in available
497                  if session.day not in faculty_daily_duties[f.id]
498                  and self._is_faculty_eligible_for_session(f.id, session,
faculty_weighted_load[f.id])
499              ]
500
501              # Sort eligible faculty by load deviation: current load + history - target
502              eligible.sort(key=lambda f: self.history_map[f.id] + faculty_weighted_load[f.id] -
self.target_loads[f.id])
503
504              # Assign up to required_invigilators
505              selected = eligible[:session.required_invigilators]
506              assignment[session.id] = [f.id for f in selected]
507
508              w_hrs = calculate_session_weighted_hours(session)
509              for f in selected:
510                  faculty_daily_duties[f.id].add(session.day)
511                  faculty_weighted_load[f.id] += w_hrs
512
513          obj = self._get_objective(assignment, faculty_weighted_load)
514          return obj, assignment
515
```

Lines / Code	Detailed Functional & Logic Explanation
Lines 480-490	Sorts sessions so that Saturday sessions (which have higher day weights) and sessions with the fewest available faculty are assigned first (MRV heuristic).
Lines 492-499	Filters faculty for eligibility, ensuring they do not exceed daily duty limits or hard category targets.
Lines 502-506	Sorts eligible faculty by load deviation (Least Cost selection) and assigns the top candidates to the session.
Lines 508-514	Updates the assigned workloads and daily duties, computes the initial objective score, and returns the greedy assignment.

Section 10: Hill Climbing Local Search (Lines 516–684)

Refines the initial schedule by running 20,000 iterations of random moves, accepting only changes that reduce the objective cost.

```
    def _run_local_search(self, initial_assignment: Dict[str, List[str]], max_iterations: int = 20000) -> Tuple[float, Dict[str, List[str]]]:
        """
        Refines the initial assignment using Hill Climbing Local Search.
        Strictly preserves daily limits, overrides, PG conflicts, and hard category loads.
        """
        random.seed(42)

        assignment = {s_id: list(facs) for s_id, facs in initial_assignment.items()}
        faculty_daily_duties = {f.id: set() for f in self.input_data.faculty_list}
        faculty_weighted_load = {f.id: 0.0 for f in self.input_data.faculty_list}

        for s_id, facs in assignment.items():
            session = next(s for s in self.sessions if s.id == s_id)
            w_hrs = calculate_session_weighted_hours(session)
            for f_id in facs:
                faculty_daily_duties[f_id].add(session.day)
                faculty_weighted_load[f_id] += w_hrs

        current_obj = self._get_objective(assignment, faculty_weighted_load)
        best_obj = current_obj
        best_assignment = {s_id: list(facs) for s_id, facs in assignment.items()}

        for _ in range(max_iterations):
            move_type = random.choice(["swap_faculty", "swap_sessions", "fill_unassigned"])

            if move_type == "swap_faculty":
                # Swap an assigned faculty member with an unassigned eligible faculty member
                session = random.choice(self.sessions)
                assigned_facs = assignment[session.id]
                if not assigned_facs:
                    continue

                f1_id = random.choice(assigned_facs)
                f2 = random.choice(self.input_data.faculty_list)
                f2_id = f2.id

                if f1_id == f2_id:
                    continue
                if session.day in faculty_daily_duties[f2_id]:
                    continue

                w_hrs = calculate_session_weighted_hours(session)

                # Check eligibility (including overrides, PG conflicts, and hard category target limits)
                if not self._is_faculty_eligible_for_session(f2_id, session, faculty_weighted_load[f2_id]):
                    continue

                # Propose swap
                faculty_weighted_load[f1_id] -= w_hrs
                faculty_daily_duties[f1_id].remove(session.day)
                faculty_weighted_load[f2_id] += w_hrs
                faculty_daily_duties[f2_id].add(session.day)

                new_obj = self._get_objective(assignment, faculty_weighted_load)

                if new_obj < current_obj:
                    assigned_facs.remove(f1_id)
```

```
                    assigned_facs.append(f2_id)
                    current_obj = new_obj
                    if current_obj < best_obj:
                        best_obj = current_obj
                        best_assignment = {s_id: list(facs) for s_id, facs in
assignment.items()}
                else:
                    # Revert
                    faculty_weighted_load[f1_id] += w_hrs
                    faculty_daily_duties[f1_id].add(session.day)
                    faculty_weighted_load[f2_id] -= w_hrs
                    faculty_daily_duties[f2_id].remove(session.day)

            elif move_type == "swap_sessions":
                # Swap duties of two faculty members between two sessions
                s1 = random.choice(self.sessions)
                s2 = random.choice(self.sessions)
                if s1.id == s2.id:
                    continue

                s1_facs = assignment[s1.id]
                s2_facs = assignment[s2.id]
                if not s1_facs or not s2_facs:
                    continue

                f1_id = random.choice(s1_facs)
                f2_id = random.choice(s2_facs)
                if f1_id == f2_id:
                    continue

                w1 = calculate_session_weighted_hours(s1)
                w2 = calculate_session_weighted_hours(s2)

                # Verify daily duty constraints
                if s2.day != s1.day:
                    if s2.day in faculty_daily_duties[f1_id]:
                        continue
                    if s1.day in faculty_daily_duties[f2_id]:
                        continue

                # Verify dynamic eligibility
                if not self._is_faculty_eligible_for_session(f1_id, s2,
faculty_weighted_load[f1_id] - w1):
                    continue
                if not self._is_faculty_eligible_for_session(f2_id, s1,
faculty_weighted_load[f2_id] - w2):
                    continue

                # Propose swap
                faculty_weighted_load[f1_id] += w2 - w1
                faculty_weighted_load[f2_id] += w1 - w2

                if s1.day != s2.day:
                    faculty_daily_duties[f1_id].remove(s1.day)
                    faculty_daily_duties[f1_id].add(s2.day)
                    faculty_daily_duties[f2_id].remove(s2.day)
                    faculty_daily_duties[f2_id].add(s1.day)

                new_obj = self._get_objective(assignment, faculty_weighted_load)

                if new_obj < current_obj:
```

```
                    s1_facs.remove(f1_id)
                    s1_facs.append(f2_id)
                    s2_facs.remove(f2_id)
                    s2_facs.append(f1_id)
                    current_obj = new_obj
                    if current_obj < best_obj:
                        best_obj = current_obj
                        best_assignment = {s_id: list(facs) for s_id, facs in
assignment.items()}
                else:
                    # Revert
                    faculty_weighted_load[f1_id] -= w2 - w1
                    faculty_weighted_load[f2_id] -= w1 - w2

                    if s1.day != s2.day:
                        faculty_daily_duties[f1_id].add(s1.day)
                        faculty_daily_duties[f1_id].remove(s2.day)
                        faculty_daily_duties[f2_id].add(s2.day)
                        faculty_daily_duties[f2_id].remove(s1.day)

            elif move_type == "fill_unassigned":
                # Try to fill an unassigned slot in a session
                session = random.choice(self.sessions)
                assigned_facs = assignment[session.id]
                if len(assigned_facs) < session.required_invigilators:
                    # Find eligible faculty who don't have duties today and satisfy hard targets
                    eligible = [
                        f.id for f in self.input_data.faculty_list
                        if session.day not in faculty_daily_duties[f.id]
                        and f.id not in assigned_facs
                        and self._is_faculty_eligible_for_session(f.id, session,
faculty_weighted_load[f.id])
                    ]
                    if eligible:
                        f2_id = random.choice(eligible)
                        w_hrs = calculate_session_weighted_hours(session)

                        # Assign
                        faculty_weighted_load[f2_id] += w_hrs
                        faculty_daily_duties[f2_id].add(session.day)
                        assigned_facs.append(f2_id)

                        new_obj = self._get_objective(assignment, faculty_weighted_load)
                        if new_obj < current_obj:
                            current_obj = new_obj
                            if current_obj < best_obj:
                                best_obj = current_obj
                                best_assignment = {s_id: list(facs) for s_id, facs in
assignment.items()}
                        else:
                            # Revert
                            faculty_weighted_load[f2_id] -= w_hrs
                            faculty_daily_duties[f2_id].remove(session.day)
                            assigned_facs.remove(f2_id)

        return best_obj, best_assignment

```

Lines / Code	Detailed Functional & Logic Explanation
Lines 538	Loops for a maximum of 20,000 iterations to perform local search optimization.
Lines 539	Randomly selects one of three move types: `swap_faculty`, `swap_sessions`, or `fill_unassigned`.
Lines 541-584	Implements `swap_faculty`: selects an assigned faculty member and swaps them with an eligible, unassigned faculty member if it reduces the objective cost.
Lines 585-649	Implements `swap_sessions`: swaps duties between two faculty members across two different sessions, checking daily limits and eligibility before verifying if the swap reduces the objective cost.
Lines 650-682	Implements `fill_unassigned`: attempts to assign an eligible, unassigned faculty member to a vacant slot if it reduces the objective cost.

Section 11: Branch-and-Bound Backtracking Solver (Lines 685–803)

Runs the backtracking algorithm to search for the global optimum, using bounding checks to prune search paths early.

```
    def solve(self, max_steps: int = 100000, run_diag: bool = True) -> AllocationResult:
        """
        Solves the invigilation scheduling optimization problem using hybrid search.
        If a perfect allocation is impossible, it returns the best feasible partial schedule.
        """
        # 1. Greedy initialization
        greedy_obj, greedy_assignment = self._run_greedy_initialization()

        # 2. Local search optimization to establish a tight bound
        best_obj, best_assignment = self._run_local_search(greedy_assignment)

        # Helper function for lower-bound calculations in backtracking search
        def get_min_contrib(h_f: float, t_f: float, L: float) -> float:
            diff = h_f + L - t_f
            contrib_sq = diff ** 2 if diff > 0 else 0.0
            contrib_worse = 0.0
            if abs(h_f) > 1e-5:
                if diff > 0:
                    worse = max(0.0, diff - abs(h_f))
                    contrib_worse = worse * 100.0
            return contrib_sq + contrib_worse

        # 3. Exact Backtracking Branch-and-Bound to guarantee optimality for full schedules
        if max_steps > 0:
            # Sort sessions: Saturday first, and sessions with fewer available faculty first
(MRV Heuristic)
            session_avail_counts = []
            for s in self.sessions:
                avail = self._get_available_faculty_for_session(s)
                session_avail_counts.append((s, len(avail)))

            sorted_sessions = [
                x[0] for x in sorted(
                    session_avail_counts,
                    key=lambda x: (x[1], -x[0].day_weight, x[0].id)
                )
            ]

            current_assignment: Dict[str, List[str]] = {s.id: [] for s in self.sessions}
            faculty_daily_duties: Dict[str, Set[int]] = {f.id: set() for f in
self.input_data.faculty_list}
            faculty_weighted_load: Dict[str, float] = {f.id: 0.0 for f in
self.input_data.faculty_list}

            # Since we search only for FULL schedules, initial lower bound is sum of min
contributions
            initial_lb = 0.0
            for f in self.input_data.faculty_list:
                initial_lb += get_min_contrib(self.history_map[f.id], self.target_loads[f.id],
0.0)

            partial_lb = [initial_lb]
            step_count = [0]
            timeout_reached = [False]

            def backtrack(session_idx: int, slot_idx: int, last_fac_idx: int):
                step_count[0] += 1
                if step_count[0] > max_steps:
                    timeout_reached[0] = True
                    return
```

```
740
741                 current_session = sorted_sessions[session_idx]
742                 req_inv = current_session.required_invigilators
743
744                 if slot_idx == req_inv:
745                     if session_idx + 1 == len(sorted_sessions):
746                         obj = self._get_objective(current_assignment, faculty_weighted_load)
747                         nonlocal best_obj, best_assignment
748                         if obj < best_obj:
749                             best_obj = obj
750                             best_assignment = {s_id: list(facs) for s_id, facs in
current_assignment.items()}
751                         return
752                     else:
753                         backtrack(session_idx + 1, 0, -1)
754                         return
755
756                 available_fac = self._get_available_faculty_for_session(current_session)
757
758                 eligible_fac = []
759                 for fac in available_fac:
760                     fac_idx = self.faculty_index[fac.id]
761                     if fac_idx <= last_fac_idx:
762                         continue
763                     if current_session.day in faculty_daily_duties[fac.id]:
764                         continue
765                     if not self._is_faculty_eligible_for_session(fac.id, current_session,
faculty_weighted_load[fac.id]):
766                         continue
767                     eligible_fac.append((fac, fac_idx))
768
769                 # Sort eligible faculty to prioritize historically underloaded / less loaded
(LCV Heuristic)
770                 eligible_fac.sort(key=lambda x: self.history_map[x[0].id] +
faculty_weighted_load[x[0].id] - self.target_loads[x[0].id])
771
772                 slot_weighted_hrs = calculate_session_weighted_hours(current_session)
773
774                 for fac, fac_idx in eligible_fac:
775                     if timeout_reached[0]:
776                         break
777
778                     old_load = faculty_weighted_load[fac.id]
779                     old_contrib = get_min_contrib(self.history_map[fac.id],
self.target_loads[fac.id], old_load)
780
781                     new_load = old_load + slot_weighted_hrs
782                     new_contrib = get_min_contrib(self.history_map[fac.id],
self.target_loads[fac.id], new_load)
783
784                     diff_contrib = new_contrib - old_contrib
785                     partial_lb[0] += diff_contrib
786
787                     if partial_lb[0] < best_obj:
788                         # Assign
789                         current_assignment[current_session.id].append(fac.id)
790                         faculty_daily_duties[fac.id].add(current_session.day)
791                         faculty_weighted_load[fac.id] += slot_weighted_hrs
792
793                         backtrack(session_idx, slot_idx + 1, fac_idx)
794
```

```
                        # Revert
                        current_assignment[current_session.id].pop()
                        faculty_daily_duties[fac.id].remove(current_session.day)
                        faculty_weighted_load[fac.id] -= slot_weighted_hrs

                    partial_lb[0] -= diff_contrib

            backtrack(0, 0, -1)

```

Lines / Code	Detailed Functional & Logic Explanation
Lines 690-694	Runs Greedy Initialization and Local Search first to establish a tight upper bound for the objective cost.
Lines 697-705	Defines `get_min_contrib`, which computes the minimum objective cost contribution for a faculty member's workload.
Lines 707-724	Sorts sessions using the MRV heuristic and initializes trackers for assignments, workloads, and daily duties.
Lines 735-754	Defines the recursive `backtrack` function, which stops when a full schedule is reached and updates the best assignment found if the objective cost is improved.
Lines 756-770	Filters available faculty for the session, sorting them by load deviation (LCV heuristic) to explore the most promising paths first.
Lines 774-785	Computes the lower bound: updates the lower bound value (`partial_lb`) for the branch. If this value is greater than or equal to the best objective cost found so far (`best_obj`), it prunes the branch and returns.
Lines 787-800	Assigns the faculty member, recurses to the next slot, and reverts the state (backtracks) if the search branch does not yield an improved solution.

Section 12: Result Compilation Engine (Lines 804–945)

Formats the final schedule and workload statistics, runs validation checks, and compiles selection justifications.

```
804         # Build results
805         schedule = [
806             SessionAllocation(session_id=s_id, assigned_faculty_ids=facs)
807             for s_id, facs in best_assignment.items()
808         ]
809 
810         # Calculate final metrics and compile reports
811         final_loads = {f.id: 0.0 for f in self.input_data.faculty_list}
812         final_hours = {f.id: 0.0 for f in self.input_data.faculty_list}
813         final_sessions = {f.id: [] for f in self.input_data.faculty_list}
814 
815         for s_alloc in schedule:
816             session = next(s for s in self.sessions if s.id == s_alloc.session_id)
817             w_hrs = calculate_session_weighted_hours(session)
818             for f_id in s_alloc.assigned_faculty_ids:
819                 final_loads[f_id] += w_hrs
820                 final_hours[f_id] += session.duration_hours
821                 final_sessions[f_id].append(session.id)
822 
823         # Run explicit validation checks
824         validation_errors = self.validate_allocation(schedule)
825 
826         coverage_errors = [e for e in validation_errors if "Coverage" in e]
827         other_errors = [e for e in validation_errors if "Coverage" not in e]
828 
829         # Determine feasibility state
830         if other_errors:
831             feasibility_status = "INFEASIBLE"
832             success = False
833         elif coverage_errors:
834             feasibility_status = "PARTIAL FEASIBLE"
835             success = False
836         else:
837             feasibility_status = "FEASIBLE"
838             success = True
839 
840         # Construct diagnostic reports if schedule is not fully successful
841         diagnostic_report = []
842         if other_errors:
843             diagnostic_report.extend(other_errors)
844         elif coverage_errors:
845             unfilled_sessions = []
846             for sa in schedule:
847                 session = next(s for s in self.sessions if s.id == sa.session_id)
848                 if len(sa.assigned_faculty_ids) < session.required_invigilators:
849                     unfilled_sessions.append(session.id)
850                     available_facs = self._get_available_faculty_for_session(session)
851                     fac_names = [f.name for f in available_facs]
852                     diagnostic_report.append(
853                         f" - Session {session.id} ({session.label}) has
{len(sa.assigned_faculty_ids)} "
854                         f"assigned out of {session.required_invigilators} required. "
855                         f"Available unique faculty for this session due to PG
conflicts/overrides: {', '.join(fac_names) if fac_names else 'None'}"
856                     )
857             if run_diag:
858                 relaxation_source = self.run_diagnostics()
859                 diagnostic_report.append(f"Diagnostics: {relaxation_source}")
860 
861         # Compile summaries and tracking statistics
```

```
862             faculty_summaries: List[FacultyReport] = []
863             worsened_count = 0
864             improved_count = 0
865             neutral_count = 0
866 
867             for f in self.input_data.faculty_list:
868                 h_f = self.history_map[f.id]
869                 a_f = final_loads[f.id]
870                 t_f = self.target_loads[f.id]
871                 c_f = h_f + a_f - t_f
872 
873                 overload = max(0.0, a_f - t_f)
874                 underload = max(0.0, t_f - a_f)
875 
876                 # Determine history impact status
877                 if abs(c_f) < abs(h_f):
878                     impact_status = "IMPROVED"
879                     improved_count += 1
880                 elif abs(c_f) > abs(h_f):
881                     impact_status = "WORSENED"
882                     worsened_count += 1
883                 else:
884                     impact_status = "NEUTRAL"
885                     neutral_count += 1
886 
887                 explanations = {}
888                 for s_id in final_sessions[f.id]:
889                     sess = next(s for s in self.sessions if s.id == s_id)
890                     reason_parts = []
891                     if h_f < 0:
892                         reason_parts.append(f"Compensated for historical underload of {h_f:+.1f}
hrs")
893                     elif h_f > 0:
894                         reason_parts.append(f"Assigned despite historical overload of {h_f:+.1f} hrs
to meet session requirements")
895                     else:
896                         reason_parts.append("Historically balanced")
897 
898                     if sess.day_weight > 1.0:
899                         reason_parts.append(f"Saturday assignment (weight {sess.day_weight:.1f})")
900                     else:
901                         reason_parts.append("Weekday assignment (weight 1.0)")
902 
903                     explanations[s_id] = ". ".join(reason_parts)
904 
905                 faculty_summaries.append(
906                     FacultyReport(
907                         faculty_id=f.id,
908                         name=f.name,
909                         category_name=f.category_name,
910                         assigned_sessions=final_sessions[f.id],
911                         assigned_hours=final_hours[f.id],
912                         assigned_weighted_load=a_f,
913                         target_load=t_f,
914                         overload=overload,
915                         underload=underload,
916                         historical_imbalance=h_f,
917                         cumulative_imbalance=c_f,
918                         impact_status=impact_status,
919                         selection_explanations=explanations
920                     )
```

```
            )

        loads_list = [final_loads[f.id] for f in self.input_data.faculty_list]
        targets_list = [self.target_loads[f.id] for f in self.input_data.faculty_list]

        jains = calculate_jains_index(loads_list, targets_list)
        gini = calculate_gini_coefficient(loads_list, targets_list)

        history_report = (
            f"History Impact Analysis: Out of {len(self.input_data.faculty_list)} faculty
members, "
            f"{improved_count} improved their load balance (closer to target), "
            f"{worsened_count} worsened, and {neutral_count} remained unchanged."
        )

        return AllocationResult(
            success=success,
            schedule=schedule,
            faculty_summaries=faculty_summaries,
            jains_fairness_index=jains,
            gini_coefficient=gini,
            feasibility_report=feasibility_status,
            conflict_report=diagnostic_report,
            history_impact_report=history_report
        )

```

Lines / Code	Detailed Functional & Logic Explanation
Lines 811-821	Calculates final loads, hours, and assignments for each faculty member.
Lines 823-838	Runs validation checks and determines the feasibility status (`FEASIBLE`, `PARTIAL FEASIBLE`, or `INFEASIBLE`).
Lines 840-860	Generates diagnostic reports for any remaining validation errors or unfilled sessions.
Lines 862-921	Compiles summaries for each faculty member, evaluating history impact status (`IMPROVED`, `WORSENED`, or `NEUTRAL`) and writing selection justifications.
Lines 923-927	Computes the final Jain's Fairness Index and Gini Coefficient metrics.
Lines 929-933	Constructs the history impact summary report.
Lines 935-944	Returns the complete `AllocationResult` object.

Section 13: Configuration Loader (Lines 946–1054)

Parses configuration data from dictionaries or JSON strings into type-safe dataclass structures.

```
946 # ======================================================================
947 # 4. CONFIGURATION LOADER
948 # ======================================================================
949
950 def load_from_dict(data: Dict[str, Any]) -> AllocationInput:
951     """
952     Parses a configuration dictionary into an AllocationInput object.
953     """
954     exam_type_str = data.get("exam_type", "midsem").lower()
955     exam_type = ExamType.MIDSEM if exam_type_str == "midsem" else ExamType.ENDSEM
956
957     # Parse categories
958     categories_raw = data.get("categories", {})
959     categories: Dict[str, FacultyCategory] = {}
960
961     if isinstance(categories_raw, list):
962         for cat in categories_raw:
963             name = cat["name"]
964             weight = float(cat.get("ratio_weight", 1.0))
965             categories[name] = FacultyCategory(name=name, ratio_weight=weight)
966     elif isinstance(categories_raw, dict):
967         for name, weight in categories_raw.items():
968             if isinstance(weight, dict):
969                 w_val = float(weight.get("ratio_weight", 1.0))
970             else:
971                 w_val = float(weight)
972             categories[name] = FacultyCategory(name=name, ratio_weight=w_val)
973     else:
974         raise ValueError("Categories must be a list or dictionary.")
975
976     # Parse faculty list
977     faculty_list_raw = data.get("faculty_list", [])
978     faculty_list: List[Faculty] = []
979     for f in faculty_list_raw:
980         fac_id = f["id"]
981         fac_name = f.get("name", fac_id)
982         cat_name = f["category"]
983         pg_blocks = f.get("pg_timetable_blocks", [])
984         overrides = f.get("availability_overrides", [])
985
986         faculty_list.append(
987             Faculty(
988                 id=fac_id,
989                 name=fac_name,
990                 category_name=cat_name,
991                 pg_timetable_blocks=pg_blocks,
992                 availability_overrides=overrides
993             )
994         )
995
996     # Default durations
997     default_duration = 2.0 if exam_type == ExamType.MIDSEM else 3.0
998
999     # Parse sessions
1000     sessions_raw = data.get("sessions", [])
1001     sessions: List[Session] = []
1002     for s in sessions_raw:
1003         s_id = s["id"]
1004         day = int(s["day"])
1005         sess_num = int(s["session_num"])
```

```
        label = s.get("label", f"Day {day} Session {sess_num}")
        req_inv = int(s["required_invigilators"])

        default_day_weight = 1.5 if day == 6 else 1.0
        day_weight = float(s.get("day_weight", default_day_weight))
        duration = float(s.get("duration_hours", default_duration))

        sessions.append(
            Session(
                id=s_id,
                day=day,
                session_num=sess_num,
                label=label,
                required_invigilators=req_inv,
                day_weight=day_weight,
                duration_hours=duration
            )
        )

    # Parse history
    history_raw = data.get("history", [])
    history: List[HistoricalRecord] = []

    if isinstance(history_raw, list):
        for h in history_raw:
            f_id = h["faculty_id"]
            val = float(h.get("previous_imbalance", 0.0))
            history.append(HistoricalRecord(faculty_id=f_id, previous_imbalance=val))
    elif isinstance(history_raw, dict):
        for f_id, val in history_raw.items():
            history.append(HistoricalRecord(faculty_id=f_id, previous_imbalance=float(val)))

    ratio_mode = data.get("category_ratio_mode", "target_load_scaling")

    return AllocationInput(
        faculty_list=faculty_list,
        categories=categories,
        sessions=sessions,
        history=history,
        exam_type=exam_type,
        category_ratio_mode=ratio_mode
    )

def load_from_json(json_str: str) -> AllocationInput:
    """Parses a JSON string into an AllocationInput object."""
    data = json.loads(json_str)
    return load_from_dict(data)

# ============================================================================
```

Lines / Code	Detailed Functional & Logic Explanation
Lines 950-955	Determines the exam type from the configuration, default parsing to `ExamType.MIDSEM` if not specified.
Lines 957-974	Parses category weights, handling both list and dictionary formats.
Lines 976-994	Parses the faculty list, mapping conflict blocks and availability overrides.

Lines / Code	Detailed Functional & Logic Explanation
Lines 996-1024	Parses sessions, applying weekend weights and duration defaults.
Lines 1026-1037	Parses historical records of workload imbalance.
Lines 1039-1047	Returns the populated `AllocationInput` object.
Lines 1049-1051	Defines `load_from_json` to load configuration data from JSON strings.

Section 14: Worsening Explanation Engine (Lines 1055–1117)

Analyzes the schedule to explain why a faculty member's historical imbalance worsened.

```
# 5. CLI RUNNER LOGIC & EXPLANATIONS
# ======================================================================

def explain_worsening(summary: FacultyReport, input_data: AllocationInput, schedule: List[SessionAllocation]) -> str:
    """
    Generates a clear explanation of why a faculty member's historical imbalance worsened,
    identifying PG conflicts, availability overrides, or general staffing limitations as causes.
    """
    h_f = summary.historical_imbalance
    c_f = summary.cumulative_imbalance

    if abs(c_f) <= abs(h_f):
        return "N/A (Imbalance improved or remained unchanged)"

    if h_f == 0:
        return "Previously perfectly balanced (0.0). Since the exam required duties to cover all sessions, any assignment deviates from 0.0."

    reasons = []
    for s_id in summary.assigned_sessions:
        session = next((s for s in input_data.sessions if s.id == s_id), None)
        if not session:
            continue

        other_facs = [fac for fac in input_data.faculty_list if fac.id != summary.faculty_id]

        # Check who was available for this session statically (no PG conflict, no override)
        available_others = []
        blocked_by_pg = []
        blocked_by_override = []
        for fac in other_facs:
            if input_data.exam_type == ExamType.MIDSEM and session.id in fac.pg_timetable_blocks:
                blocked_by_pg.append(fac.name)
            elif session.id in fac.availability_overrides:
                blocked_by_override.append(fac.name)
            else:
                available_others.append(fac)

        # Of those available, who was already assigned on this day?
        assigned_others_on_day = []
        for fac in available_others:
            for sa in schedule:
                if sa.session_id != session.id:
                    other_sess = next((s for s in input_data.sessions if s.id == sa.session_id), None)
                    if other_sess and other_sess.day == session.day and fac.id in sa.assigned_faculty_ids:
                        assigned_others_on_day.append(fac.name)
                        break

        eligible_others = [fac for fac in available_others if fac.name not in assigned_others_on_day]

        if len(eligible_others) < session.required_invigilators:
            details = []
            if blocked_by_pg:
                details.append(f"{len(blocked_by_pg)} PG conflicts")
            if blocked_by_override:
```

```
                details.append(f"{len(blocked_by_override)} overrides")
            if assigned_others_on_day:
                details.append(f"{len(assigned_others_on_day)} assigned on same day")
            reasons.append(f"Session {session.id} ({session.label}): shortage of other eligible faculty (" + ", ".join(details) + ")")

    if not reasons:
        return "Assigned to balance category ratio requirements and overall department load."
    return "Required on specific days due to constraints: " + "; ".join(reasons)

```

Lines / Code	Detailed Functional & Logic Explanation
Lines 1058-1070	Checks if the imbalance actually worsened. If it improved, remained unchanged, or was historically balanced, it returns a placeholder explanation.
Lines 1073-1100	Checks the availability of all other faculty members for each session assigned to the target faculty member, identifying conflicts and day-wise assignments.
Lines 1102-1116	If there is a shortage of eligible alternatives on those days, it constructs an explanation detailing the constraints (e.g. PG conflicts, overrides, same-day assignments) that forced the assignment.

Section 15: CLI Reporting & Main Entry Loop (Lines 1118–1293)

Defines sample data for testing, formats and prints the terminal report, and handles CLI execution.

```
SAMPLE_DATA = {
    "exam_type": "midsem",
    "category_ratio_mode": "target_load_scaling",
    "categories": [
        {"name": "Professor", "ratio_weight": 2.0},
        {"name": "Associate Professor", "ratio_weight": 3.0},
        {"name": "Assistant Professor", "ratio_weight": 4.0}
    ],
    "faculty_list": [
        {"id": "P1", "name": "Prof. P1", "category": "Professor", "pg_timetable_blocks": [],
"availability_overrides": []},
        {"id": "P2", "name": "Prof. P2", "category": "Professor", "pg_timetable_blocks": [],
"availability_overrides": []},
        {"id": "AS1", "name": "Assoc Prof. AS1", "category": "Associate Professor",
"pg_timetable_blocks": [], "availability_overrides": []},
        {"id": "AS2", "name": "Assoc Prof. AS2", "category": "Associate Professor",
"pg_timetable_blocks": [], "availability_overrides": []},
        {"id": "AS3", "name": "Assoc Prof. AS3", "category": "Associate Professor",
"pg_timetable_blocks": [], "availability_overrides": []},
        {"id": "AP1", "name": "Asst Prof. AP1", "category": "Assistant Professor",
"pg_timetable_blocks": [], "availability_overrides": []},
        {"id": "AP2", "name": "Asst Prof. AP2", "category": "Assistant Professor",
"pg_timetable_blocks": [], "availability_overrides": []},
        {"id": "AP3", "name": "Asst Prof. AP3", "category": "Assistant Professor",
"pg_timetable_blocks": [], "availability_overrides": []},
        {"id": "AP4", "name": "Asst Prof. AP4", "category": "Assistant Professor",
"pg_timetable_blocks": [], "availability_overrides": []}
    ],
    "sessions": [
        {"id": "D11", "day": 1, "session_num": 1, "label": "Monday FN", "required_invigilators":
4},
        {"id": "D12", "day": 1, "session_num": 2, "label": "Monday AN", "required_invigilators":
3},
        {"id": "D21", "day": 2, "session_num": 1, "label": "Tuesday FN",
"required_invigilators": 3},
        {"id": "D22", "day": 2, "session_num": 2, "label": "Tuesday AN",
"required_invigilators": 3},
        {"id": "D31", "day": 3, "session_num": 1, "label": "Wednesday FN",
"required_invigilators": 2},
        {"id": "D32", "day": 3, "session_num": 2, "required_invigilators": 4},
        {"id": "D41", "day": 4, "session_num": 1, "required_invigilators": 3},
        {"id": "D42", "day": 4, "session_num": 2, "required_invigilators": 2},
        {"id": "D51", "day": 5, "session_num": 1, "required_invigilators": 2},
        {"id": "D52", "day": 5, "session_num": 2, "required_invigilators": 2},
        {"id": "D61", "day": 6, "session_num": 1, "required_invigilators": 2, "day_weight":
1.5},
        {"id": "D62", "day": 6, "session_num": 2, "required_invigilators": 1, "day_weight": 1.5}
    ],
    "history": [
        {"faculty_id": "P1", "previous_imbalance": 0.0},
        {"faculty_id": "P2", "previous_imbalance": 0.0},
        {"faculty_id": "AS1", "previous_imbalance": 0.0},
        {"faculty_id": "AS2", "previous_imbalance": 0.0},
        {"faculty_id": "AS3", "previous_imbalance": 0.0},
        {"faculty_id": "AP1", "previous_imbalance": 0.0},
        {"faculty_id": "AP2", "previous_imbalance": 0.0},
        {"faculty_id": "AP3", "previous_imbalance": 0.0},
        {"faculty_id": "AP4", "previous_imbalance": 0.0}
    ]
}
```

```
1163
1164  def print_report(result: AllocationResult, input_data: AllocationInput):
1165      print("=" * 100)
1166      print("                      UNIVERSITY INVIGILATION DUTY ALLOCATION REPORT")
1167      print("=" * 100)
1168      print(f"Feasibility Status:     {result.feasibility_report}")
1169      print(f"Jain's Fairness Index:  {result.jains_fairness_index:.4f} (1.0 is perfectly
equal/fair)")
1170      print(f"Gini Coefficient:       {result.gini_coefficient:.4f} (0.0 is perfect equality)")
1171      print("-" * 100)
1172
1173      # Print Day-wise Allocation Table
1174      print("\nDAY-WISE ALLOCATION SCHEDULE")
1175      print("-" * 100)
1176      header = f"{'Session':<10} | {'Day':<4} | {'Label':<15} | {'Req':<4} | {'Assigned Faculty'}"
1177      print(header)
1178      print("-" * 100)
1179
1180      id_to_fac_name = {f.id: f.name for f in input_data.faculty_list}
1181
1182      for sess_alloc in result.schedule:
1183          sess = next(s for s in input_data.sessions if s.id == sess_alloc.session_id)
1184          fac_names = [id_to_fac_name[f_id] for f_id in sess_alloc.assigned_faculty_ids]
1185
1186          # Mark unassigned count if session is understaffed
1187          missing_count = sess.required_invigilators - len(sess_alloc.assigned_faculty_ids)
1188          if missing_count > 0:
1189              fac_names.append(f"({missing_count} UNASSIGNED SLOTS)")
1190
1191          fac_str = ", ".join(fac_names)
1192          print(f"{sess.id:<10} | Day {sess.day:<2} | {sess.label:<15} |
{sess.required_invigilators:<4} | {fac_str}")
1193      print("-" * 100)
1194
1195      # Print Faculty-wise Workload Summary Table
1196      print("\nFACULTY WORKLOAD SUMMARY")
1197      print("-" * 100)
1198      fac_header = (
1199          f"{'Faculty ID':<10} | {'Name':<15} | {'Category':<15} | "
1200          f"{'Duties':<6} | {'Hours':<6} | {'W_Load':<7} | {'Target':<7} | {'Imbalance':<9} |
{'Status'}"
1201      )
1202      print(fac_header)
1203      print("-" * 100)
1204
1205      for summary in sorted(result.faculty_summaries, key=lambda s: s.faculty_id):
1206          duties_count = len(summary.assigned_sessions)
1207          imb_str = f"{summary.cumulative_imbalance:+.2f}"
1208          print(
1209              f"{summary.faculty_id:<10} | {summary.name:<15} | {summary.category_name:<15} | "
1210              f"{duties_count:<6} | {summary.assigned_hours:<6.1f} | "
1211              f"{summary.assigned_weighted_load:<7.2f} | {summary.target_load:<7.2f} | "
1212              f"{imb_str:<9} | {summary.impact_status}"
1213          )
1214      print("-" * 100)
1215      print(result.history_impact_report)
1216      print("-" * 100)
1217
1218      # Diagnostics / Conflict logging if not completely successful
1219      if not result.success or "PARTIAL" in result.feasibility_report:
1220          print("\nDIAGNOSTIC & CONFLICT REPORT")
```

```
1221          print("-" * 100)
1222          for line in result.conflict_report:
1223              print(f"  - {line}")
1224          print("-" * 100)
1225
1226      # Print Worsening Explanations table if any faculty worsened
1227      worsened_reports = [s for s in result.faculty_summaries if s.impact_status == "WORSENED"]
1228      if worsened_reports:
1229          print("\nHISTORICAL IMBALANCE WORSENING EXPLANATIONS")
1230          print("-" * 100)
1231          print(f"{'Faculty ID':<10} | {'Name':<15} | {'Imbalance':<9} | {'Justification / Explanation'}")
1232          print("-" * 100)
1233          for summary in worsened_reports:
1234              explanation = explain_worsening(summary, input_data, result.schedule)
1235              print(f"{summary.faculty_id:<10} | {summary.name:<15} | {summary.cumulative_imbalance:+.2f} | {explanation}")
1236          print("-" * 100)
1237
1238      # Print explanations for selection (sample logs)
1239      print("\nSELECTION EXPLANATION (SAMPLE LOG)")
1240      print("-" * 100)
1241      printed_count = 0
1242      for summary in sorted(result.faculty_summaries, key=lambda s: s.faculty_id):
1243          if summary.assigned_sessions:
1244              print(f"Faculty: {summary.name} ({summary.category_name})")
1245              for s_id, explanation in summary.selection_explanations.items():
1246                  print(f"  - Session {s_id}: {explanation}")
1247              printed_count += 1
1248              if printed_count >= 5:  # Limit explanation output length
1249                  print("  ... (truncated for brevity) ...")
1250                  break
1251      print("=" * 100)
1252
1253  def main():
1254      if len(sys.argv) > 1:
1255          config_path = sys.argv[1]
1256          print(f"Loading configuration from file: {config_path}")
1257          if not os.path.exists(config_path):
1258              print(f"Error: File '{config_path}' not found.")
1259              sys.exit(1)
1260          try:
1261              with open(config_path, 'r') as f:
1262                  config_data = json.load(f)
1263          except Exception as e:
1264              print(f"Error parsing JSON file: {e}")
1265              sys.exit(1)
1266      else:
1267          print("No configuration file provided. Running default sample data (PDF Example)...")
1268          config_data = SAMPLE_DATA
1269
1270          sample_filename = "sample_config.json"
1271          try:
1272              with open(sample_filename, "w") as f:
1273                  json.dump(SAMPLE_DATA, f, indent=4)
1274              print(f"Created '{sample_filename}' in workspace for reference.")
1275          except Exception as e:
1276              print(f"Warning: Could not create '{sample_filename}': {e}")
1277
1278      input_data = load_from_dict(config_data)
1279
```

```
    # Run the solver using ratio mode from config
    ratio_mode = config_data.get("category_ratio_mode", "target_load_scaling")
    solver = InvigilationSolver(input_data, ratio_mode=ratio_mode)

    start_time = time.time()
    result = solver.solve()
    end_time = time.time()

    print_report(result, input_data)
    print(f"Solved in {((end_time - start_time) * 1000):.2f} ms")

if __name__ == "__main__":
    main()
```

Lines / Code	Detailed Functional & Logic Explanation
Lines 1118-1162	Defines a complete mock dataset (`SAMPLE_DATA`) representing faculty, sessions, and categories for default runs.
Lines 1164-1252	Defines `print_report` to format and print the day-wise allocation table, faculty workload summary, history impact report, and worsening explanations.
Lines 1253-1277	The `main` function checks for a command-line configuration file. If none is provided, it falls back to the default mock dataset and saves it to `sample_config.json`.
Lines 1278-1290	Loads the configuration, runs the solver, prints the report, and logs the total execution time.